CS 224n 作业 #2：Word2Vec 与依存句法分析

截止日期：4月18日，星期四，太平洋标准时间下午4:30。
在本作业中，你将复习 Word2Vec 背后的数学原理，并使用 PyTorch 构建一个神经依存句法分析器。如需复习 PyTorch 基础知识，请查看 Canvas 上的 PyTorch 复习课程。在第1部分中，你将探索使用朴素 softmax 损失训练 Word2vec 模型所涉及的偏导数。在第2部分中，你将了解两种通用的神经网络技术（Adam 优化和 Dropout）。在第3部分中，你将使用第2部分中的技术实现并训练一个依存句法分析器，然后分析一些错误的依存句法分析结果。

如果你使用 LaTeX，可以使用 \ifans{} 来输入你的解答。
请在 Gradescope 上正确标注题目，否则如果你不标注题目，助教将会扣分。

1. 理解 word2vec（15分）

回想一下，word2vec 背后的关键洞见是“一个词的含义由其周围的词决定”。具体来说，考虑一个“中心”词 c，其前后被一定长度的上下文所包围。我们将这个上下文窗口中的词称为“外部词”（O）。例如，在图1中，上下文窗口长度为2，中心词 c 是“banking”，外部词是“turning”、“into”、“crises”和“as”：

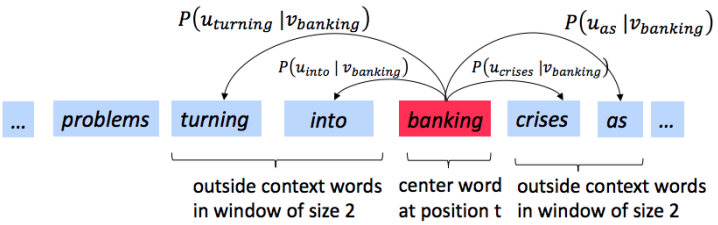


图1：窗口大小为2的word2vec跳元预测模型

跳元word2vec旨在学习概率分布 $P(O|C)$。具体来说，给定一个特定词 o 和一个特定词 c，我们希望预测 $P(O=o|C=c)$：即词 o 是 c 的“外部”词的概率（即它落在 c的上下文窗口内）。我们通过对一系列向量点积应用softmax函数来建模这一概率：

$$P(O=o \mid C=c)=\frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w\in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \tag{1}$$

对于每个词，我们学习向量 u 和 v，其中 $\mathbf{u}_o$ 是表示外部词 o的“外部”向量，$\mathbf{v}_c$ 是表示中心词 c的“中心”向量。我们将这些参数存储在两个矩阵中，$\mathbf{U}$ 和 $\mathbf{V}$。$\mathbf{U}$ 的列是所有“外部”向量 $\mathbf{u}_w$；$\mathbf{V}$ 的列是所有“中心”向量 $\mathbf{v}_w$。$\mathbf{U}$ 和 $\mathbf{V}$ 都为每个 w ∈词汇表[1]包含一个向量。

回顾课堂内容，对于一对单词 c 和 o，损失函数为：

$$\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U}) = -\log P(O=o|C=c). \tag{2}$$

[1]假设我们词汇表中的每个词都对应一个整数编号 k。加粗的小写字母表示向量。$\mathbf{u}_k$ 既是 $\mathbf{U}$ 的第 k^{th} 列，也是索引为 k的词的外部词向量。$\mathbf{v}_k$ 既是 $\mathbf{V}$ 的第 k^{th} 列，也是索引为 k的词的中心词向量。**为了简化记号，我们将交替使用 k 来指代词 k 以及词的索引 k。**

我们可以将此损失视为真实分布$\mathbf{y}$ 与预测分布$\hat{}\mathbf{y}$之间的交叉熵[2] ，针对特定的中心词 c 和特定的外部词 o。这里，$\mathbf{y}$ 和$\hat{}\mathbf{y}$ 都是长度等于词汇表单词数的向量。此外，这些向量中的第 k^{th} 个条目表示在给定 c 的情况下，第 k^{th} 个单词作为“外部词”的条件概率。真实经验分布$\mathbf{y}$ 是一个独热向量，对于这个特定的中心词 c 和外部词 o.[3] 的示例，真实外部词 o处为 1，其他位置均为 0。预测分布$\hat{}\mathbf{y}$ 是我们在方程 (1) 中给出的概率分布 $P(O|C = c)$ 。

注意：在本作业中，计算导数时，请使用课堂上复习的方法（即不使用泰勒级数近似）。

[2]真实（离散）概率分布 p 与另一个分布 q 之间的**交叉熵损失**为 $-\sum_i p_i \log(q_i)$。[3]请注意，整个训练数据集中上下文词的真实条件概率分布并非独热编码。

(a)（2分）证明朴素softmax损失（公式2）与$\mathbf{y}$ 和$\hat{}\mathbf{y}$之间的交叉熵损失相同，即（注意$\mathbf{y}$ （真实分布）、$\hat{}\mathbf{y}$ （预测分布）是向量，而$\hat{\mathbf{y}}_o$ 是标量）：

$$-\sum_{w\in \text{Vocab}} \mathbf{y}_w \log(\hat{\mathbf{y}}_w) = -\log(\hat{\mathbf{y}}_o). \tag{3}$$

你的答案应写为一行。你可以用文字描述你的答案。

(b)（6 分） i. 计算 $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ 关于 $\mathbf{v}_c$ 的偏导数。 请用 $\mathbf{y}$、$\hat{}\mathbf{y}$ 和 $\mathbf{U}$ 来表示你的答案，并展示你的解题过程以获得满分。

- **注意**：你对偏导数的最终答案应遵循形状约定：任何函数 $f(x)$ 关于 x 的偏导数应具有与 x **相同的形状**。[4]

- 请以向量化形式给出你对偏导数的答案。 例如，当我们要求你用 $\mathbf{y}$、$\hat{}\mathbf{y}$ 和$\mathbf{U}$ 来表示你的答案时，你不得引用这些项在最终答案中的具体元素（例如 $\mathbf{y}_1$ $\mathbf{y}_2$ $\dots$）、、。

ii. 你计算出的梯度何时等于零?

提示： 你可能需要回顾并使用一些线性代数入门概念。

iii. 你求得的梯度是两项之差。请解释当该梯度从词向量 v 中减去时，这两项各自如何改进词向量。

c.

(c)（1分）在许多使用词嵌入的下游应用中，会使用L2归一化向量（例如$\mathbf{u}/||\mathbf{u}||_2$，其中$||\mathbf{u}||_2 = \sqrt{\sum_i u_i^2}$）而非其原始形式（例如$\mathbf{u}$）。让我们考虑一个假设的下游任务：将短语二分类为正面或负面，其中你根据各个词嵌入之和来决定符号。L2归一化何时会丢失对下游任务有用的信息？何时不会?

提示： 考虑以下情况：对于某些词 $\mathbf{u}_x = \alpha \mathbf{u}_y$， $x \neq y$ 以及某个标量 α。当 α 为正时，归一化后的 $\mathbf{u}_x$ 和归一化后的 $\mathbf{u}_y$? 的值将是多少? $\mathbf{u}_x$ 和 $\mathbf{u}_y$ 可能如何相关联，以使这种归一化影响或不影响最终的分类结果?

(d)（5 分）计算 $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ 关于每个“外部”词向量 $\mathbf{u}_w$'s 的偏导数。将有两种情况：当 $w = o$ 为真实的“外部”词向量时，以及 $w \neq o$ 为所有其他词时。请用 $\mathbf{y}$、$\hat{}\mathbf{y}$ 和 $\mathbf{v}_c$ 表示你的答案。在本小题中，你也可以使用这些项中的特定元素（例如 $\mathbf{y}_1$ $\mathbf{y}_2$ $\dots$），，。注意 $\mathbf{u}_w$ 是向量，而 $\mathbf{y}_1$ $\mathbf{y}_2$ ..，，。是标量。请展示你的工作以获得满分。

(e)（1分）写出 $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ 关于 $\mathbf{U}$ 的偏导数。请用列向量 $\frac{\partial \mathbf{J}(\mathbf{v}_c, o, \mathbf{U})}{\partial \mathbf{u}_1}, \frac{\partial \mathbf{J}(\mathbf{v}_c, o, \mathbf{U})}{\partial \mathbf{u}_2}, \dots, \frac{\partial \mathbf{J}(\mathbf{v}_c, o, \mathbf{U})}{\partial \mathbf{u}_{|\text{Vocab}|}}$ 来分解你的答案。 No 推导是必要的，只需以矩阵形式给出答案即可。

[4]这使我们能够使用梯度下降高效地最小化函数，而无需担心重塑或维度不匹配的问题。遵循形状约定，我们可以保证 $\theta := \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$ 是一个定义良好的更新规则。

2. 机器学习与神经网络（8分）

(a)（4分）Adam优化器

回顾标准随机梯度下降更新规则：

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \alpha \nabla_{\boldsymbol{\theta}_t} J_{\text{minibatch}}(\boldsymbol{\theta}_t)$$

其中 $t+1$ 是当前时间步，$\boldsymbol{\theta}$ 是包含所有模型参数的向量，（$\boldsymbol{\theta}_t$ 是时间步 t 时的模型参数，$\boldsymbol{\theta}_{t+1}$ 是时间步 $t+1$ 时的模型参数），J 是损失函数，$\nabla_{\boldsymbol{\theta}} J_{\text{minibatch}}(\boldsymbol{\theta})$ 是损失函数关于小批量数据上参数的梯度，α 是学习率。Adam 优化[5] 使用一种更复杂的更新规则，包含两个额外步骤。[6]

i.（2分）首先，Adam 使用了一种称为**动量** 的技巧，通过跟踪$\mathbf{m}$（梯度的滚动平均值）来实现：

$$\mathbf{m}_{t+1} \leftarrow \beta_1 \mathbf{m}_t + (1-\beta_1) \nabla_{\boldsymbol{\theta}_t} J_{\text{minibatch}}(\boldsymbol{\theta}_t)$$
$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \alpha \mathbf{m}_{t+1}$$

其中 β_1 是介于 0 和 1 之间的超参数（通常设为 0.9）。请用 2–4 句话简要解释（无需数学证明，只需给出直观理解）使用 $\mathbf{m}$ 如何阻止更新变化过大，以及为什么这种低方差总体上可能有助于学习。

ii.（2 分）Adam 通过跟踪 $\mathbf{v}$（梯度幅值的滚动平均值），以 **自适应学习率** 的技巧扩展了 **动量** 的思想：

$$\mathbf{m}_{t+1} \leftarrow \beta_1 \mathbf{m}_t + (1-\beta_1) \nabla_{\boldsymbol{\theta}_t} J_{\text{minibatch}}(\boldsymbol{\theta}_t)$$
$$\mathbf{v}_{t+1} \leftarrow \beta_2 \mathbf{v}_t + (1-\beta_2)(\nabla_{\boldsymbol{\theta}_t} J_{\text{minibatch}}(\boldsymbol{\theta}_t) \odot \nabla_{\boldsymbol{\theta}_t} J_{\text{minibatch}}(\boldsymbol{\theta}_t))$$
$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \alpha \mathbf{m}_{t+1} / \sqrt{\mathbf{v}_{t+1}}$$

其中 $\odot$ 和 / 分别表示逐元素乘法和除法（因此 $\mathbf{z} \odot \mathbf{z}$ 是逐元素平方），而 β_2 是介于 0 和 1 之间的超参数（通常设为 0.99）。由于 Adam 将更新除以 $\sqrt{\mathbf{v}}$，哪些模型参数会获得更大的更新？为什么这可能有助于
学习？

(b)（4 分）Dropout[7] 是一种正则化技术。在训练期间，dropout 以概率 p_{drop} 随机将隐藏层 $\mathbf{h}$ 中的单元置零（每个小批次丢弃不同的单元），然后将 $\mathbf{h}$ 乘以常数 γ。我们可以将其写为：

$$\mathbf{h}_{\text{drop}} = \gamma \mathbf{d} \odot \mathbf{h}$$

其中 $\mathbf{d} \in \{0,1\}^{D_h}$（$D_h$ 是 $\mathbf{h}$ 的大小）是一个掩码向量，其中每个条目以概率 p_{drop} 为 0，以概率 $(1-p_{\text{drop}})$ 为 1。γ 的选择使得 $\mathbf{h}_{\text{drop}}$ 的期望值为 $\mathbf{h}$：

$$\mathbb{E}_{p_{\text{drop}}}[\mathbf{h}_{\text{drop}}]_i = h_i$$

对于所有 $i \in \{1, \ldots, D_h\}$。

i.（2 分）γ 必须等于什么（用 p_{drop}？ 表示）？ 简要说明你的理由或展示你的计算过程使用上述方程进行推导。

ii.（2 分）为什么在训练期间应该应用 dropout？ 为什么 dropo 但 **不**应在评估期间应用？ **提示：** 查看链接的 dropout 论文可能会有所帮助。

[5]Kingma 和 Ba，2015，https://arxiv.org/pdf/1412.6980.pdf [6]实际的 Adam 更新使用了一些不太重要的额外技巧，但我们在这里不加以考虑。如果你想了解更多，可以查看：http://cs231n.github.io/neural-networks-3/#sgd [7]Srivastava 等人，2014，https://www.cs.toronto.edu/~hinton/absps/JMLRdropout.pdf

3. 基于神经网络的转移式依存句法分析（54 分）

在本节中，你将实现一个基于神经网络的依存句法分析器，目标是在 UAS（无标签依存准确率）指标上实现性能最大化。

在开始之前，请按照 README 安装本作业所需的所有依赖项。我们将使用来自 https://pytorch.org/get-started/locally/ 的 PyTorch 2.1.2，CUDA 选项设置为 None，以及 tqdm 包——它会在整个训练过程中生成进度条可视化。PyTorch 官方网站是一个很好的资源，其中包含用于理解 PyTorch 张量库和神经网络的教程。

依存句法分析器分析句子的语法结构，建立中心词 与修饰这些中心词的词之间的关系。依存句法分析器有多种类型，包括转移式分析器、基于图的分析器和基于特征的分析器。你的实现将是一个转移式分析器，它会一次一步地增量构建句法分析。在每一步中，它都会维护一个部分句法分析，其表示如下：

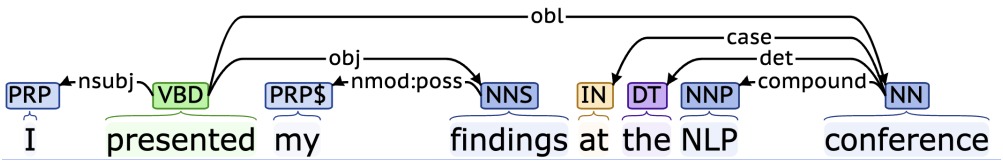
- 当前正在处理的词的栈 。
- 一个尚待处理的词缓冲区。
- 解析器预测的依存关系列表。

初始时，栈中仅包含 ROOT，依赖列表为空，缓冲区按顺序包含句子中的所有单词。在每一步中，解析器对部分解析结果应用一个转换，直到其缓冲区为空且栈大小为 1。可以应用以下转换：

- SHIFT：从缓冲区中移除第一个词并将其压入栈中。
- LEFT-ARC：将栈上的第二个（第二近添加的）项标记为第一个项的依存项，并从栈中移除第二个项，向依存关系列表中添加一个第一个词 → 第二个词的依存关系。
- RIGHT-ARC：将栈上的第一个（最近添加的）项标记为第二个项的依存项，并从栈中移除第一个项，向依存关系列表中添加一个第二个词 →第一个词 的依存关系。

在每一步中，你的解析器将使用神经网络分类器在三种转移操作中做出选择。

(a)（4分）完成解析句子 *"I presented myfindings at the NLP conference"*所需的转换序列。该句子的依存树如下所示。在每一步中，给出栈和缓冲区的配置，以及该步应用了什么转换、添加了什么新依存关系（如果有）。前三个步骤作为示例提供如下。



栈	缓冲区	新依存关系	转换
[ROOT]	[我，在，NLP，会议上，展示了，我的，研究结果]		初始配置
[ROOT，我]	[展示了，我的，研究结果，在，NLP，会议上]		移进
[ROOT，我，展示了]	[我的, 发现, 在, 这个, NLP, 会议]		SHIFT
[ROOT, 展示了]	[我的, 发现, 在, 这个, NLP, 会议]	展示了→我	LEFT-ARC

(b)（2 分）一个包含 n 个单词的句子将被解析多少步（用 n 表示）？ 请用 1–2 句话简要解释原因。

(c)（6 分）实现 `__init__` 以及 parser transitions.py 中 PartialParse 类的解析步骤函数。这实现了你的解析器将使用的转换机制。你可以通过运行 python parser transitions.py part c 来运行基本（非穷尽）测试。

(d)（8 分）我们的网络将预测下一个应应用于部分解析的转换。我们可以用它来解析单个句子，通过应用预测的转换直到解析完成。然而，神经网络在同时对 批次 数据进行预测时运行效率要高得多（即，同时为任何不同的部分解析预测下一个转换）。我们可以使用以下算法在小批次中解析句子。

算法 1小批量依存句法分析

输入： sentences，一个待分析的句子列表；model，我们用于做出分析决策的模型

将部分分析初始化为一个 PartialParses 列表，sentences 中的每个句子对应一个 PartialParses
将未完成分析初始化为部分分析的浅拷贝
当 未完成分析不为空时 **执行**
  取未完成分析中的前 batch size 个分析作为一个小批量
  使用模型为小批量中的每个部分解析预测下一个转换
  使用预测的转换对小批量中的每个部分解析执行一个解析步骤
  从尚未完成的解析中移除已完成的解析（空缓冲区和大小为1的栈）
结束循环

返回： 部分解析中每个（现已完成的）解析的依存关系。

在 parser transitions.py 中的 minibatch parse 函数中实现该算法。你可以通过运行 python parser transitions.py part d 来运行基本（非穷尽）测试。

注意：你需要正确实现小批量解析，才能评估你在***(e)***部分构建的模型。但是，你不需要它来训练模型，因此即使小批量解析尚未实现，你也应该能够完成***(e)***部分的大部分内容。

(e)（20 分）我们现在要训练一个神经网络，使其在给定栈、缓冲区和依存关系状态的情况下，预测接下来应该应用哪个转移操作。

首先，模型提取一个表示当前状态的特征向量。我们将使用原始神经依存句法分析论文中提出的特征集：*A Fast and Accurate Dependency Parser using Neural Networks*。[8] 提取这些特征的函数已在 utils/parser utils.py 中为你实现。该特征向量由一组词元组成（例如，栈中的最后一个词、缓冲区中的第一个词、栈中倒数第二个词的依存词（如果存在的话）等）。

如果存在的话，等等。它们可以表示为一个整数列表 $\mathbf{w} = [w_1, w_2, \ldots, w_m]$，其中 m 是特征的数量，每个 $0 \leq w_i < |V|$ 是词表中某个词元的索引（$|V|$ 是词表大小）。然后我们的网络为每个词查找一个嵌入，并将它们拼接成一个单一的输入向量：

$$\mathbf{x} = [\mathbf{E}_{w_1}, ..., \mathbf{E}_{w_m}] \in \mathbb{R}^{dm}$$

其中 $\mathbf{E} \in \mathbb{R}^{|V| \times d}$ 是一个嵌入矩阵，每一行 $\mathbf{E}_w$ 作为特定词 w 的向量

[8] Chen 和 Manning，2014，https://nlp.stanford.edu/pubs/emnlp2014-depparser.pdf

维度为 d。然后我们将预测计算为：

$$\begin{aligned}\mathbf{h} &= \mathrm{ReLU}(\mathbf{x}\mathbf{W} + \mathbf{b}_1)\\ \mathbf{l} &= \mathbf{h}\mathbf{U} + \mathbf{b}_2\\ \hat{\mathbf{y}} &= \mathrm{softmax}(l)\end{aligned}$$

其中 $\mathbf{h}$ 被称为隐藏层，$\mathbf{l}$ 被称为 logits，$\hat{\mathbf{y}}$ 被称为预测值，且 $\mathrm{ReLU}(z) = \max(z, 0)$。我们将训练模型以最小化交叉熵损失：

$$J(\theta) = CE(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{j=1}^{3} \mathbf{y}_j \log \hat{\mathbf{y}}_j$$

其中 $\mathbf{y}_j$ 表示 $\mathbf{y}$ 的第 j 个元素。为了计算训练集上的损失，我们将该$J(\theta)$在所有训练样本上取平均。

i. 计算 $\mathbf{h} = \mathrm{ReLU}(\mathbf{x}\mathbf{W} + \mathbf{b}_1)$ 关于 $\mathbf{x}$ 的导数。为简单起见，你只需展示某个索引 i 和 j 的导数 $\frac{\partial h_i}{\partial x_j}$。你可以忽略导数在 0 处未定义的情况。

ii. 回顾第 1b 部分，我们计算了 $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ 的偏导数。同样地，请计算 $J(\theta)$ 关于 $\mathbf{l}$ 的第 i 个元素的偏导数，记为 $\mathbf{l}_i$。具体来说，计算 $\frac{\partial CE(\mathbf{y},\hat{\mathbf{y}})}{\partial \mathbf{l}_i}$，假设 $\mathbf{l} \in \mathbb{R}^3$，$\hat{\mathbf{y}} \in \mathbb{R}^3$，$\mathbf{y} \in \mathbb{R}^3$，且真实标签为 c。

提示：你可能还记得第 1a 部分中的内容，$\frac{\partial CE(\mathbf{y},\hat{\mathbf{y}})}{\partial \mathbf{l}_i} = \sum_j \frac{\partial CE(\mathbf{y},\hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}_j} \frac{\partial \hat{\mathbf{y}}_j}{\partial \mathbf{l}_i}$，以及 $\frac{\partial CE(\mathbf{y},\hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}_j} = 0$ 如果 $j \neq c$。

iii. 我们将使用 UAS 分数作为评估指标。UAS 指的是无标签依存准确率（Unlabeled Attachment Score），其计算方式为正确预测的依存关系数量与总依存关系数量之比，不考虑关系类型（我们的模型不预测关系类型）。

在 parser model.py 中，你将找到使用 PyTorch 实现这个简单神经网络的骨架代码。完成 init、embedding lookup 和 forward 函数以实现该模型。然后完成 run.py 文件中的 train for epoch 和 train 函数。

最后执行 python run.py 来训练你的模型，并在来自 Penn Treebank（使用 Universal Dependencies 标注）的测试数据上计算预测结果。

注意：

- 对于本次作业，你需要实现线性层和嵌入层。请**不要**在代码中使用**torch.nn.Linear** 或 **torch.nn.Embedding** 模块，否则本题将会被扣分。
- 请遵循我们 TODO 中的命名要求（如果有的话），例如，如果有关于变量名的明确要求，你必须遵守这些要求才能获得满分。如果没有明确要求，你可以自由声明其他变量名。

提示：

- 要求你声明的每个变量（self.embed to hidden weight、self.embed to hidden bias、self.hidden to logits weight、self.hidden to logits bias）都对应上述变量之一（$\mathbf{W}$、$\mathbf{b}_1$、$\mathbf{U}$、$\mathbf{b}_2$）。
- 从算法中反向推导（从 $\hat{\mathbf{y}}$ 开始）并跟踪矩阵/向量的大小可能会有所帮助。
- 一旦你实现了嵌入查找（e）或前向传播（f），你可以使用标志 -e 或 -f 或两者来调用 python parser model.py，以分别对每一项进行健全性检查

函数。这些健全性检查相当基础，通过它们并不意味着你的代码没有错误。

• 调试时，可以添加调试标志：python run.py -d。这会使代码在数据的一小部分子集上运行，从而缩短模型训练时间。调试完成后，请务必移除 -d 标志以运行完整模型。

• 在调试模式下运行时，你应该能够在开发集上获得小于 0.2 的损失和大于 65 的 UAS（尽管在极少数情况下你的结果可能更低，因为训练时存在一些随机性）。

• 在整个训练数据集上训练模型（即禁用调试模式时）最多需要 **15 分钟**。

• 禁用调试模式后，你应该能够在训练集上获得小于 0.08 的损失，并在开发集上获得大于 87 的未标记附着分数。作为对比，原始神经依存句法分析论文中的模型获得了 92.5 UAS。如果需要，你可以调整模型的超参数（隐藏层大小、Adam 的超参数、训练轮数等）以提高性能（但并非必须这样做）。

交付内容：

• 在 parser transitions.py 中实现神经依存解析器所使用的转换机制，并确保其可正常运行。 -

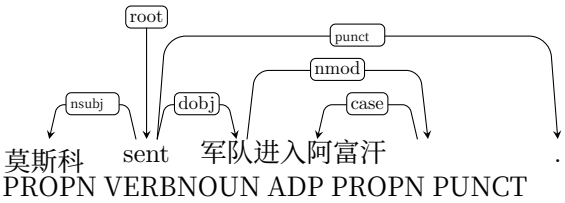
• 在 parser transitions.py 中实现小批量依存解析，并确保其可正常运行。

• 在 parser model.py 中实现神经依存解析器，并确保其可正常运行。〔我们将在评分时查看并运行此代码）。

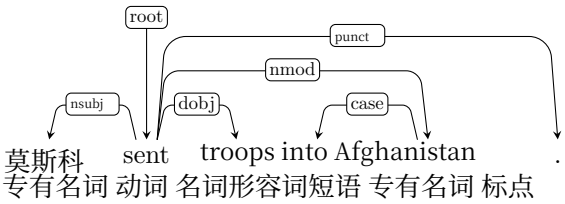
• 在 run.py 中实现训练函数，并确保其可正常运行。（我们将在评分时查看并运行此代码）。

•**在书面提交中报告你的模型在开发集上达到的最佳 UAS 以及在测试集上达到的 UAS。**

(f)（12分）我们想看看依存分析示例，并理解像我们这样的解析器可能在哪些地方出错。例如，在这个句子中：



短语*into Afghanistan* 的依存关系是错误的，因为该短语应该修饰*sent* （如*sent into Afghanistan*），而不是*troops* （因为*troops into Afghanistan* 说不通，除非莫名其妙地有一些支持阿富汗的军队）。以下是正确的句法分析：



更一般地说，解析错误有以下四种类型：

• **介词短语附着错误**：在上面的例子中，短语进入阿富汗是一个介词短语[9]。介词短语附着错误是指介词

[9]关于介词短语的示例，请参见：https://www.grammarly.com/blog/prepositional-phrase/

短语附着到了错误的核心词上（在此例中，部队是错误的核心词，而派遣才是正确的核心词）。介词短语的更多例子包括用一块石头、在午夜之前和在地毯下面。

- **动词短语附着错误**：在句子*Leaving the store unattended, I went outside to watch the parade*中，短语*leaving the store unattended* 是一个动词短语[10]。一个动词短语附着错误是指动词短语附着到了错误的核心词上（在此例中，正确的核心词是*went*）。

- **修饰语附着错误**：在句子*I am extremely short*中，副词*extremely* 是形容词*short*的修饰语。修饰语附着错误是指修饰语附着到了错误的核心词上（在此例中，正确的核心词是*short*）。

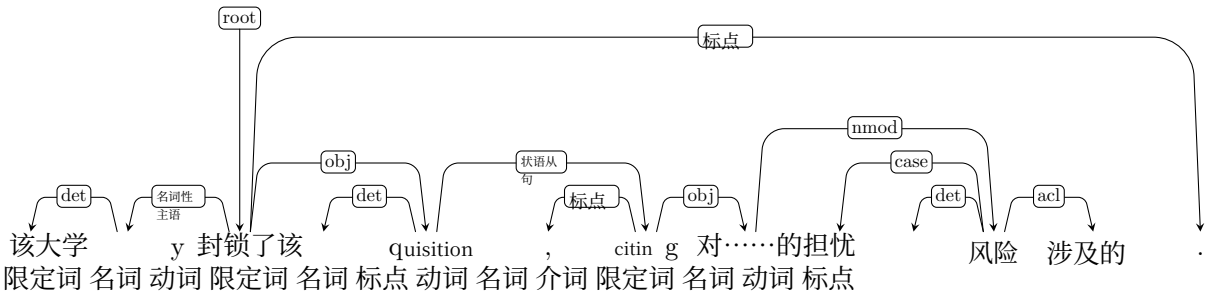
- **并列附着错误**：在句子*Would you like brown rice or garlic naan?*中，短语*brown rice* 和*garlic naan* 都是并列项，而单词*or* 是并列连词。第二个并列项（此处为*garlic naan*）应附着到第一个并列项（此处为*brown rice*）上。并列附着错误是指第二个并列项附着到了错误的核心词上（在此例中，正确的核心词是*rice*）。其他并列连词包括*and*、*but* 和*so*。

本题中有四个句子，其依存句法分析结果由解析器获得。每个句子有一种错误类型，且上述四种错误类型各出现一次。对于每个句子，请指出错误类型、错误的依存关系以及正确的依存关系。虽然每个句子应具有唯一的错误类型，但某些句子可能存在多个可能的正确依存关系。为作示范：对于上述示例，你应写为：

- **错误类型**：介词短语附着错误
- **错误的依存关系**：troops →Afghanistan
- **正确的依存关系**：sent → Afghanistan

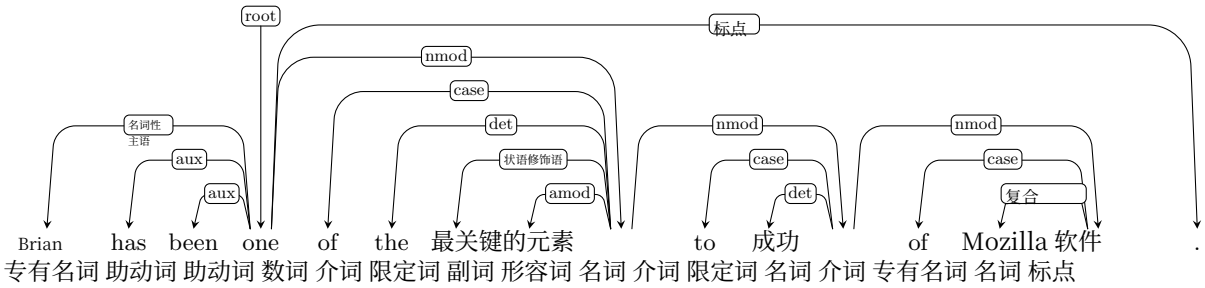
注意：依存标注有许多细节和惯例。如果你想了解更多，可以查看 *UD* 网站：*http://universaldependencies. org*[11] 或查看以下简短的入门幻灯片：*http://people.cs.georgetown.edu/nschneid/ p/UD-for-English.pdf*。请注意，你不需要了解所有这些细节就能完成这道题。在每种情况下，我们问的都是短语的依附关系，只要看它们是否修饰正确的中心词就足够了。特别是，你不需要查看依存边上的标签——只看边本身就足够了。

i.

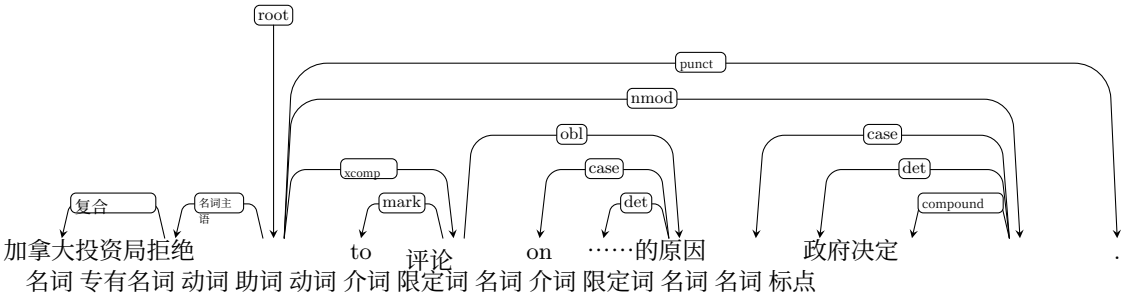


ii.

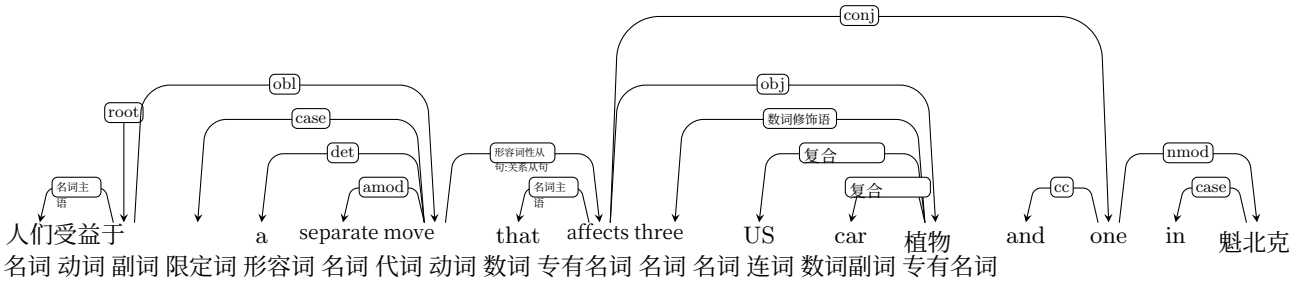
[10]关于动词短语的示例，请参见：https://examples.yourdictionary.com/verb-phrase-examples.html[11]但请注意，在作业中我们实际上使用的是 UDv1，请参见：http://universaldependencies.org/docsv1/



iii.



iv.



(g)（2分）回顾(e)部分，解析器使用的特征包括单词及其词性（POS）标签。请解释在解析器中使用词性标签作为特征的好处?

提交说明

你需要在GradeScope上以两次提交的形式提交本次作业——一次提交至“Assignment 2 [coding]”，另一次提交至“Assignment 2 [written]”：

1. 运行collect submission.sh脚本以生成你的assignment2.zip文件。
2. 将你的 assignment2.zip 文件上传到 GradeScope 的“Assignment 2 [coding]”。
3. 将你的书面解答上传到 GradeScope 的“Assignment 2 [written]”。